

Университет ИТМО

Расширенный курс по backend-разработке в среде Node.js

Лабораторная работа №3

Контейнеризация микросервисного приложения средствами Docker

Платформа для фитнес-тренировок и здоровья

Выполнила: Яковенко Ксения
Группа: БР1.1

2026 г.

1. Цель и постановка задачи

Цель работы - контейнеризировать API Gateway и каждый микросервис, собрать инфраструктуру в общем docker-compose.yml и настроить сетевое взаимодействие контейнеров.

Согласно заданию для каждого сервиса подготовлен отдельный Dockerfile. Compose поднимает восемь Node.js-приложений, PostgreSQL и RabbitMQ в общей bridge-сети.

2. Состав контейнерного приложения

Контейнер	Порт внутри сети	Назначение
api-gateway	3000	Единая точка входа и маршрутизация.
identity-service	3001	Пользователи, профили и JWT.
workout-catalog-service	3002	Каталог тренировок.
training-plan-service	3003	Тренировочные планы и RabbitMQ publisher.
progress-service	3004	Сессии тренировок и метрики.
content-service	3005	Блог и категории.
media-service	3006	Файлы и их метаданные.
notification-service	3007	Уведомления и RabbitMQ consumer.
postgres	5432	Семь отдельных логических баз.
rabbitmq	5672 / 15672	AMQP и web-интерфейс управления.

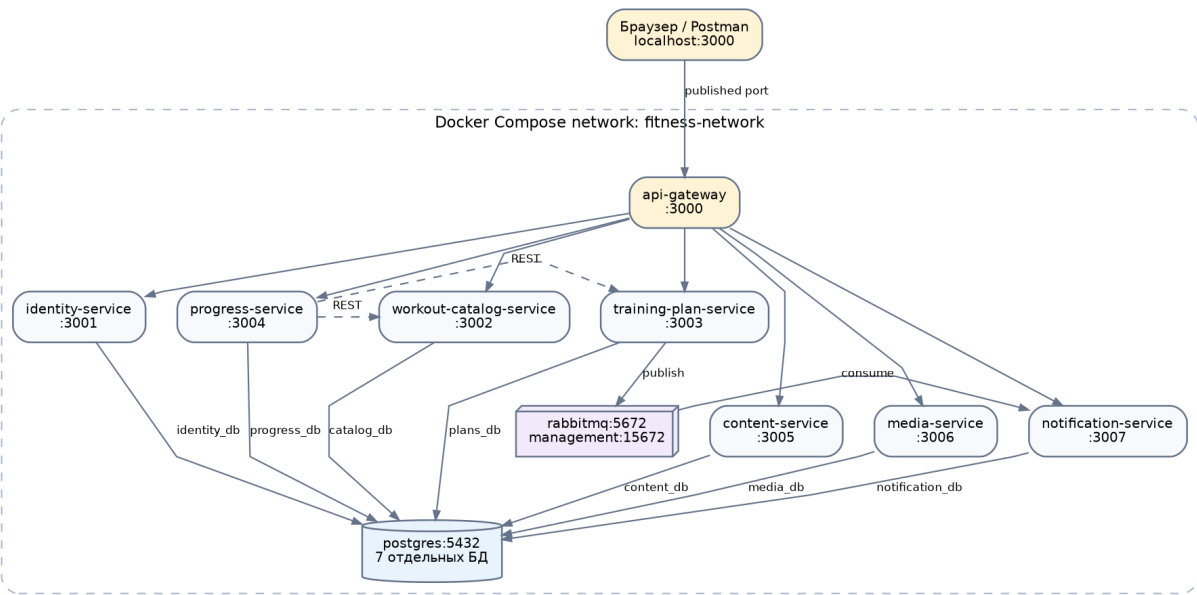


Рисунок 1 - Контейнеры и взаимодействие в fitness-network

3. Dockerfile для каждого сервиса

В каталоге `docker` находятся отдельные Dockerfile для API Gateway и семи доменных сервисов. Дополнительно создан `seed.Dockerfile` для одноразовой загрузки демонстрационных данных.

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package.json ./
RUN npm install --no-audit --no-fund
COPY tsconfig.json ./
COPY src ./src
RUN npm run build

FROM node:20-alpine AS runtime
WORKDIR /app
ENV NODE_ENV=production
COPY package.json ./
RUN npm install --omit=dev --no-audit --no-fund
COPY --from=build /app/dist ./dist
USER nodeapp
CMD ["node", "dist/<service>/app.js"]
```

Multi-stage build отделяет компиляцию TypeScript от runtime-образа. Во втором этапе устанавливаются только production dependencies. Процессы запускаются непривилегированным пользователем `nodeapp`.

4. Общий `docker-compose.yml`

Compose описывает сборку всех сервисов, переменные окружения, порты, healthcheck-и, зависимости, volumes и общую сеть `fitness-network`.

Раздел Compose	Применение
<code>build</code>	Для каждого сервиса указан собственный Dockerfile.
<code>environment</code>	Внутренние URL, DB credentials, JWT и RabbitMQ URL.
<code>depends_on</code>	Ожидание PostgreSQL, RabbitMQ и зависимых сервисов.
<code>healthcheck</code>	Проверка готовности БД, брокера и HTTP /health.
<code>networks</code>	Все контейнеры подключены к <code>fitness-network</code> .
<code>volumes</code>	Сохранение PostgreSQL, RabbitMQ и uploads.
<code>ports</code>	Доступ с хоста к Gateway, сервисам и management UI.

5. Сетевое взаимодействие

Внутри контейнера `localhost` указывает на сам контейнер. Поэтому сервисы обращаются друг к другу по DNS-именам, которые Docker Compose создаёт из названий `services`.

Локальный запуск	Docker Compose
<code>http://localhost:3001</code>	<code>http://identity-service:3001</code>
<code>localhost:15432</code>	<code>postgres:5432</code>
<code>amqp://...@localhost:5672</code>	<code>amqp://...@rabbitmq:5672</code>
<code>http://localhost:3007</code>	<code>http://notification-service:3007</code>

Ключевой принцип: published ports нужны хосту, а контейнеры взаимодействуют через внутренние порты и DNS Compose.

6. Порядок запуска и healthcheck

PostgreSQL проверяется командой `pg_isready`, RabbitMQ - `rabbitmq-diagnostics ping`. Node.js-контейнеры проверяют собственный endpoint `/health`. API Gateway запускается после того, как все доменные сервисы стали `healthy`.

1. PostgreSQL создаёт семь пользователей и семь БД через `init-multiple-databases.sh`.
2. RabbitMQ запускается с `management plugin` и учебной учётной записью `fitness`.
3. Доменные сервисы подключаются к своим БД и создают таблицы.
4. Training Plan и Notification подключаются к RabbitMQ.
5. После `healthcheck`-ов запускается API Gateway.
6. Seed запускается отдельной командой и завершает работу после заполнения данных.

7. Volumes и сохранение данных

Volume	Данные
<code>fitness_lab3_postgres_data</code>	Таблицы и записи семи БД.
<code>fitness_lab3_rabbitmq_data</code>	Состояние брокера, <code>durable queues</code> и сообщения.
<code>fitness_lab3_uploads</code>	Файлы, сохранённые Media Service.

Команда `docker compose down` останавливает контейнеры, но сохраняет volumes. Команда `docker compose down -v` удаляет тестовые данные и используется для полностью чистого повторного запуска.

8. Запуск и проверка

```
docker compose up --build -d
docker compose ps
docker compose --profile tools run --rm seed
```

После запуска проверяются следующие адреса:

- `http://localhost:3000/health/services` - состояние всех микросервисов;
- `http://localhost:3000/api/workouts` - данные Catalog Service;
- `http://localhost:3000/api/training-plans` - данные Training Plan Service;
- `http://localhost:3000/api/blog/posts` - данные Content Service;
- `http://localhost:15672` - RabbitMQ Management.

```
docker compose logs -f
docker compose logs -f training-plan-service notification-service rabbitmq
```

9. Безопасность и ограничения учебной среды

Аспект	Решение / ограничение
Права процесса	Node.js запускается пользователем nodeapp.
Секреты	В Compose указаны учебные значения; production требует secrets/Vault.
Базы	Логически разделены, но размещены на одном PostgreSQL-контейнере.
Схема БД	Для учебного запуска используется synchronize=true; production требует миграций.
Сеть	Сервисы находятся в общей bridge-сети; наружу опубликованы порты для защиты.
Наблюдаемость	Доступны HTTP healthcheck-и и docker compose logs.

10. Результат

В результате для каждого сервиса подготовлен отдельный multi-stage Dockerfile, а общий docker-compose.yml описывает сборку, сеть, инфраструктуру, порядок запуска, healthcheck-и и постоянные volumes. Контейнеры используют внутренние DNS-имена Compose, поэтому микросервисное взаимодействие не зависит от localhost. TypeScript-код прошёл статическую компиляцию, конфигурации YAML и Postman JSON прошли синтаксическую проверку.